

DOCUMENT DE CONCEPTION TECHNIQUE

Conception *technique* de la solution.

Architecture, modèles de données et contrats d'API d'une plateforme e-commerce B2B sécurisée et conteneurisée.

ÉQUIPE PROJET

Arthur L'AFFÉTER
Backend Tom LEPRIEUR
Frontend & modèle de
données Tiago DA
COSTA Intégrations API
& chatbot IA

CLASSE

CPI-3 26 DEV A
Bachelor 3 — Développement

RÉFÉRENCE

DCT — VI.0
Avril 2026

INDEX

Table des *matières*

·	Répartition des contributions	03
I	Objet du document	05
II	Périmètre technique	06
III	Stack technique & justifications	07
IV	Architecture globale	09
V	Architecture backend	10
VI	Architecture frontend	12
VII	Modèle de données	14
VIII	Contrats API	16
IX	Sécurité technique	19
X	Qualité logicielle	20
XI	Déploiement & exploitation	21
XII	Évolutivité & roadmap	22
XIII	Décisions techniques notables	23
XIV	Limites & écarts à traiter	24

ÉQUIPE PROJET

Qui a fait *quoi*.

Le projet a été conduit à trois, chacun responsable d'un domaine technique distinct mais coordonné. Cette page formalise la répartition des contributions présentée en soutenance.



BACKEND

Arthur L'AFFÉTER

API, sécurité, architecture .NET 8

- Mise en place de la **Clean Architecture** (Domain / Application / DAL / Web).
- **Composition root** Program.cs et configuration en dix sections (DI, Swagger, CORS, pipeline).
- **Authentification JWT** avec refresh tokens à rotation, politiques Admin / AuthenticatedUser.
- Validation des DTO via **FluentValidation**, mapping entités ↔ DTO via **AutoMapper**.
- Middleware **ErrorHandling** centralisé, mapping des exceptions vers codes HTTP standardisés.
- Implémentation des **controllers REST** (auth, catalogue, panier, commandes, admin) et logique métier associée.
- Tests unitaires backend **xUnit + Moq + FluentAssertions**.

FRONTEND & DONNÉES

Tom LEPRIEUR

SPA Angular, UX, schéma relationnel

- Architecture **Angular 21** — structure core / features / shared, lazy loading par module.
- **Routage et guards** (authGuard, adminGuard, guestGuard), interceptors JWT et refresh.
- Composants UI des parcours métier : **catalogue, fiche produit, panier, checkout, compte**.
- Internationalisation **ngx-translate** (fr / en / ar / he), support RTL.
- Conception du **modèle relationnel** SQL Server — entités, relations, contraintes d'intégrité, indexes.
- Migrations **EF Core** versionnées et stratégie de **snapshots** OrderItem (intégrité historique).
- Modélisation factures / avoirs (immutabilité, numérotation automatique).

ÉQUIPE PROJET — SUITE

Intégrations & IA.



INTÉGRATIONS & IA

Tiago DA COSTA

Services externes, conteneurisation, LLM

- **Conteneurisation Docker Compose** — orchestration des cinq services (frontend, backend, SQL, Ollama, LibreTranslate).
- Intégration **Stripe** : PaymentIntent, SetupIntent, tokenisation des cartes, webhooks.
- Intégration **Cloudinary** : upload, transformations à la volée, distribution CDN des médias.
- Mise en place du **chatbot IA local** — Ollama + Mistral, prompts contextuels au domaine.
- Service de **traduction dynamique** auto-hébergé (LibreTranslate) et endpoints `/api/translate`.
- Endpoints `/api/contact/chatbot` et persistance des conversations.
- Documentation interactive **Swagger / OpenAPI** publiée à `/swagger`.

I. Objet du document

Ce **Document de Conception Technique (DCT)** décrit l'architecture complète de la solution e-commerce **Althea Systems**. Il constitue la référence technique permettant à toute équipe IT de comprendre, maintenir, faire évoluer ou reproduire le système.

« Le DCT formalise les choix d'architecture qui structurent la plateforme — du conteneur à la requête HTTP — et constitue la base de référence pour la maintenance et les évolutions futures. »

Le document couvre

- l'**architecture globale** et les choix technologiques justifiés ;
- l'architecture logicielle **frontend** et **backend** ;
- le **modèle de données** relationnel ;
- les **contrats d'API** exposés ;
- la stratégie de **sécurité** ;
- les procédures de **déploiement** ;
- la **maintenance** et l'**évolutivité** de la solution.

§

8

SERVICES ORCHESTRÉS

14

SECTIONS DU DCT

4

LANGUES SUPPORTÉES

II. Périmètre technique

2.1 — Applications couvertes

APPLICATION	TECHNOLOGIE	PORT LOCAL
Front office web (SPA responsive)	Angular 21	:4200
Backoffice administrateur (SPA)	Angular 21	:4200/admin
API backend REST	ASP.NET Core (.NET 8)	:5000
Base de données relationnelle	SQL Server	:1433
Stockage images	Cloudinary (SaaS)	—
Paieement en ligne	Stripe (SaaS)	—
Chatbot IA local	Ollama + Mistral	:11434
Traduction dynamique	LibreTranslate	:5001

2.2 — Objectifs techniques

- Code **modulaire et maintenable** (Clean Architecture).
- Séparation claire des **responsabilités** entre couches.
- Sécurisation des accès — **JWT**, rôles, validation.
- Support de **charge évolutif**.
- Déploiement **reproductible** via conteneurs Docker.
- Onboarding accéléré pour les futures équipes.



L'ensemble est conçu autour d'une **API REST unique** servant à la fois le front office et le backoffice. Cette unification réduit la surface technique à maintenir et garantit une cohérence des contrats entre les deux interfaces.

III. Stack technique & justifications

Trois grands ensembles structurent la solution : **frontend Angular 21**, **backend ASP.NET Core 8**, et un ensemble d'**intégrations externes** conteneurisées via Docker Compose.

3.1 — Frontend · Angular 21

BIBLIOTHÈQUE	USAGE	JUSTIFICATION
Angular 21	Framework SPA	Typage fort (TypeScript), modularité, CLI puissant, écosystème mature.
RxJS	Flux asynchrones	Natif Angular, observable pattern pour API calls et état applicatif.
Tailwind CSS	Styles utilitaires	Développement rapide, responsive intégré, pas de CSS custom à maintenir.
ngx-translate	Internationalisation (i18n)	Standard Angular, support RTL, chargement lazy des fichiers de traduction.
Stripe.js	Intégration paiement	Stripe Elements PCI-compliant, formulaire sécurisé côté client.
Chart.js	Graphiques admin	Léger, bien intégré Angular, varié (histogramme, camembert).

3.2 — Backend · ASP.NET Core (.NET 8)

BIBLIOTHÈQUE	USAGE	JUSTIFICATION
ASP.NET Core	Framework Web API	Performance, écosystème Microsoft, support Clean Architecture native.
Entity Framework Core	ORM	Requêtes paramétrées (protection SQLi), migrations versionnées.
FluentValidation	Validation des DTO	Règles déclaratives, messages d'erreur standardisés.
AutoMapper	Mapping entités ↔ DTO	Séparation domain/presentation, code réduit.
JWT Bearer	Authentification	Standard OAuth2, stateless, support des rôles.

3.3 — Infrastructure et intégrations externes

SERVICE	USAGE	JUSTIFICATION
Docker Compose	Orchestration locale	Environnements reproductibles, onboarding simplifié.
SQL Server	BDD relationnelle	Intégrité transactionnelle, support EF Core complet.
Cloudinary	Stockage et CDN images	Transformations à la volée, CDN mondial, API REST simple.
Stripe	Paieement sécurisé	Conformité PCI-DSS niveau 1, webhooks fiables.
Ollama + Mistral	Chatbot LLM local	Données maîtrisées, pas de coût API externe, réponses contextuelles.
LibreTranslate	Traduction dynamique	Open-source, auto-hébergé, support multilingue dont RTL.

Synthèse des choix structurants

Découplage

Toute intégration externe est exposée derrière une **interface de service**, ce qui permet de remplacer un prestataire sans refactor majeur.

Maîtrise

Le LLM tourne **en local** via Ollama : pas de fuite de données, coût d'exploitation marginal, latence prédictible.

Conformité

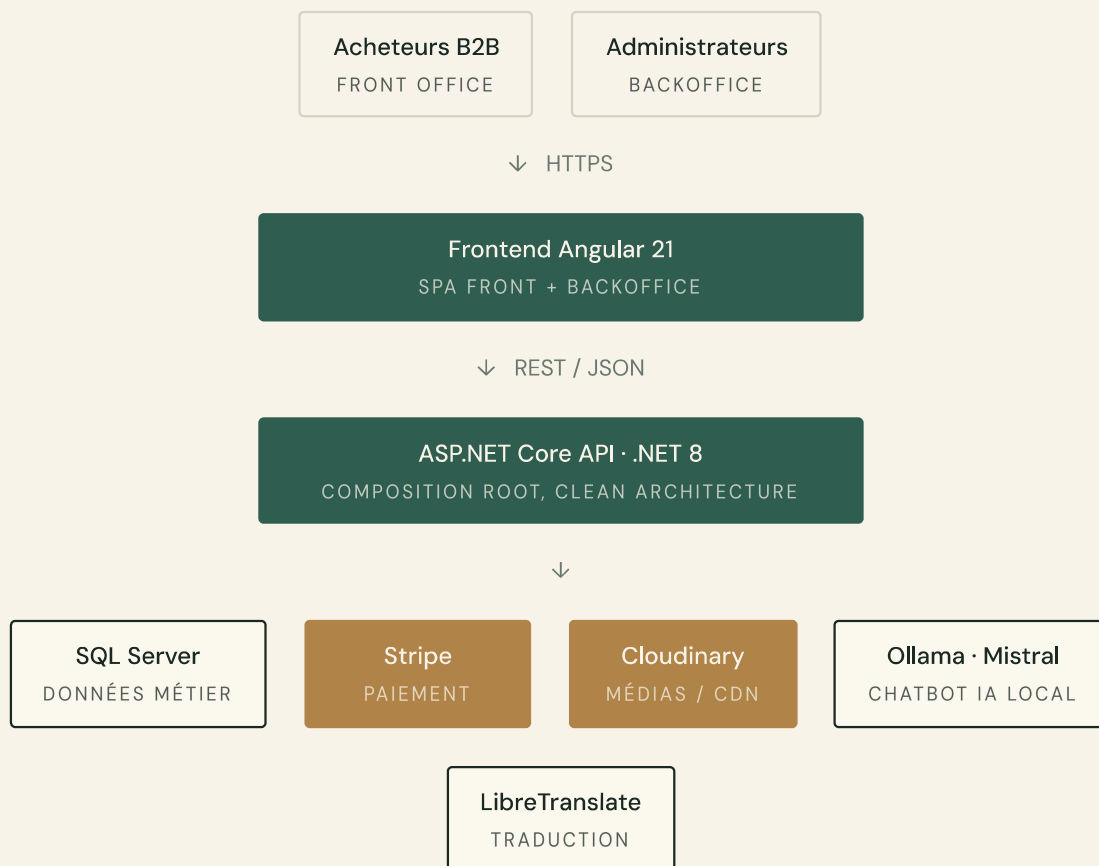
L'externalisation du paiement à Stripe garantit la **conformité PCI-DSS** sans alourdir l'infrastructure interne.

Reproductibilité

Tout l'environnement (BDD, IA, traduction) se relance via une seule commande `docker compose up`.

IV. Architecture globale

4.1 — Schéma de haut niveau



4.2 — Vue de déploiement (Docker Compose)

```
services:
  frontend:    image: node / nginx    ports: "80:80"
  backend:     image: .NET 8          ports: "5000:5000"
  sqlserver:   image: mssql/server    ports: "1433:1433"
  ollama:      image: ollama/ollama   ports: "11434:11434"
```

⌘ Tous les services partagent le même **réseau Docker**. Stripe et Cloudinary sont consommés en HTTPS sortant ; les autres composants restent **internes** à l'hôte.

V. Architecture backend

5.1 — Clean Architecture · Couches et responsabilités

```
src/  
├─ Domain/           ← Entités, value objects, exceptions, interfaces repositories  
├─ Application/      ← Services métier, DTOs, AutoMapper, FluentValidation  
├─ DAL/              ← AppDbContext, repositories EF Core, migrations SQL Server  
├─ Authentication/   ← JwtService, refresh tokens, policies (Admin, AuthenticatedUser)  
├─ ErrorHandling/    ← Middleware global → HTTP status codes, ErrorResponse standardisé  
└─ Web/              ← Controllers REST, Program.cs (DI / 10 sections), Swagger, CORS
```

Flux de dépendances. Web → Application + DAL + Authentication + ErrorHandling · Application → Domain · Domain n'a *aucune dépendance externe*.

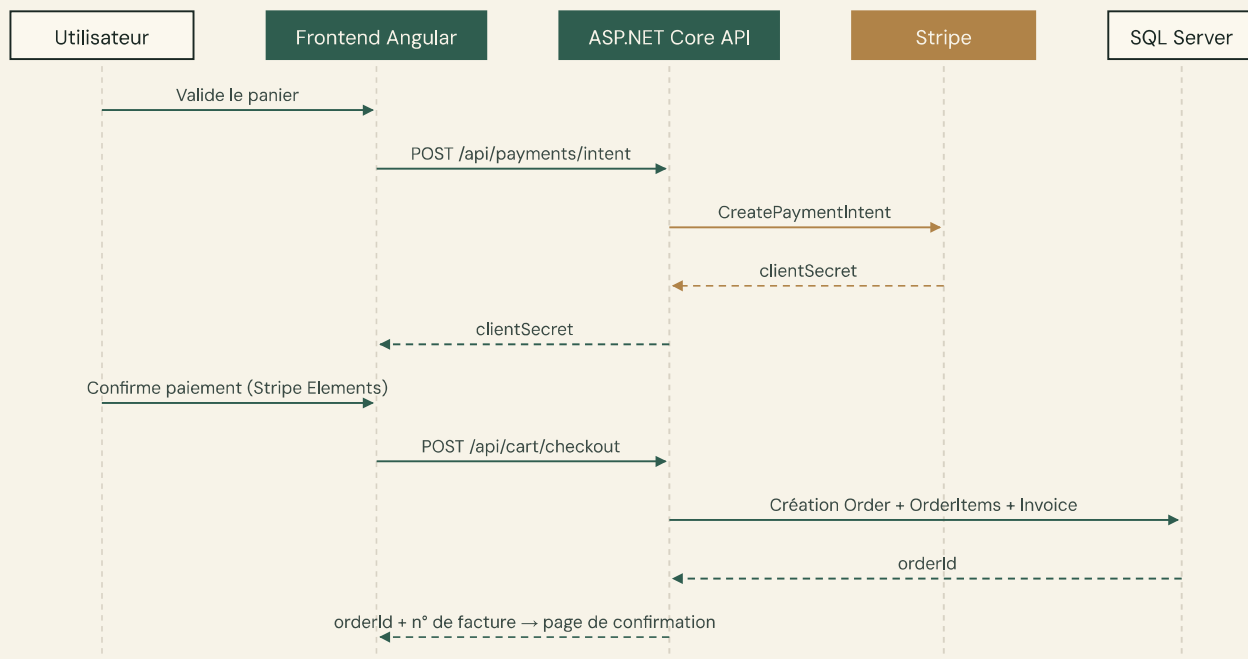
5.2 — Composition root · **Program.cs**

La configuration principale est centralisée dans src/Web/Program.cs en dix sections successives :

- | | |
|-------------------------------------|--|
| 1 Services métier (Application) | 6 Middleware de gestion d'erreurs |
| 2 DbContext SQL Server (DAL) | 7 Services Cloudinary |
| 3 Auth JWT + Authorization policies | 8 Services Stripe |
| 4 Swagger / OpenAPI | 9 Services Ollama / LibreTranslate |
| 5 CORS | 10 Pipeline HTTP (ordre des middlewares) |

5.3 — Séquence checkout

Diagramme d'enchaînement décrivant l'orchestration entre l'utilisateur, le frontend, l'API, Stripe et la base de données lors de la validation d'une commande.



! L'**idempotence** du checkout est garantie côté API : la confirmation Stripe doit aboutir avant l'écriture définitive de la commande, et un *webhook* Stripe consolide l'état en cas de désynchronisation.

VI. Architecture frontend

6.1 — Structure applicative

```
src/app/
├─ core/          ← Services HTTP, guards (auth/admin/guest), interceptors JWT, état auth
├─ features/      ← Modules pages métier
│   ├─ home/      ← Accueil (carrousel, catégories, top produits)
│   ├─ catalog/    ← Catalogue par catégorie
│   ├─ product/    ← Fiche produit + produits similaires
│   ├─ search/     ← Recherche avancée (facettes + tri)
│   └─ cart/       ← Panier dynamique
```

6.2 — Routage principal

ROUTE	ACCÈS	DESCRIPTION
/	Public	Page d'accueil
/categories/:slug	Public	Catalogue d'une catégorie
/produits/:slug	Public	Fiche produit
/recherche	Public	Recherche avancée
/panier	Public	Panier (invité + connecté)
/checkout	Authentié	Processus de commande
/contact	Public	Formulaire + chatbot
/mon-compte/*	Authentié	Espace utilisateur
/admin/*	Admin	Backoffice
/login, /register	Invité	Authentification
/forgot-password, /reset-password	Invité	Réinitialisation mot de passe

6.3 — Protection des routes (Guards)

GUARD	CONDITION	REDIRECTION
authGuard	Utilisateur connecté requis	→ /login
adminGuard	Rôle admin requis	→ /
guestGuard	Réservé aux visiteurs non connectés	→ /

Principes UX appliqués

- Découpage **features-first** : chaque domaine fonctionnel possède son module dédié, isolant ses services et composants.
- **Lazy loading** systématique des modules non-critiques (admin, account, checkout).
- **Interceptors HTTP** centralisant l'injection du JWT, la gestion du refresh, et l'unification des erreurs API.
- Internationalisation **RTL-ready** avec ngx-translate (fr / en / ar / he).



Le panier supporte un **mode invité** via un identifiant UUID stocké côté client. Lors de la connexion, le panier est *fusionné* avec celui de l'utilisateur authentifié.

VII. Modèle de données

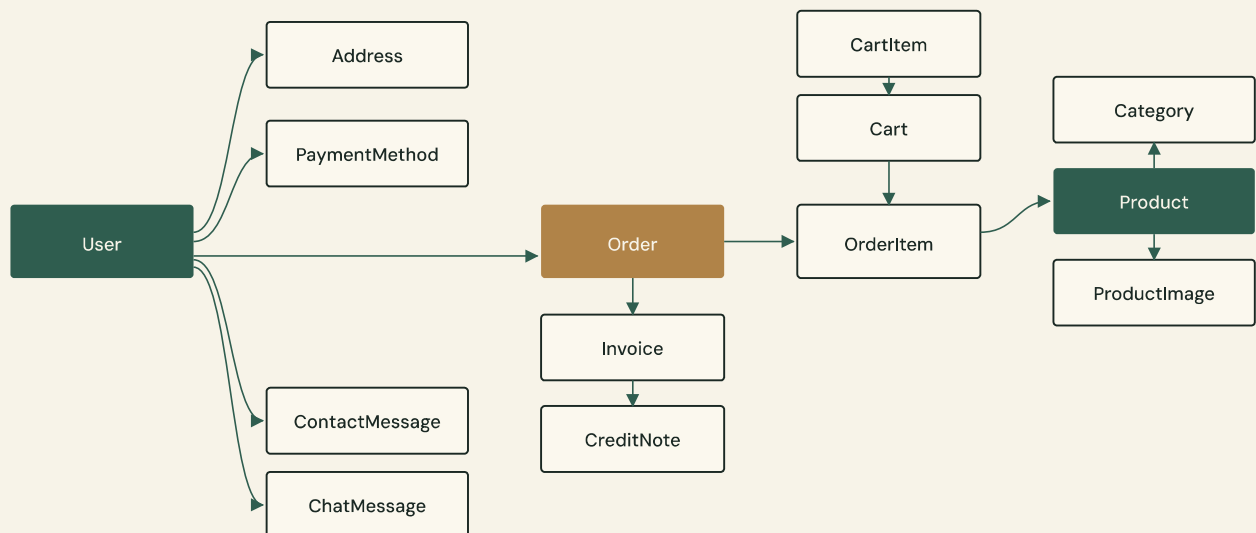
7.1 — Entités principales

ENTITÉ	DESCRIPTION	POINTS CLÉS
User	Compte utilisateur	Email unique, rôle, statut (actif/inactif/en attente)
Address	Adresse facturation/livraison	Liée à User, plusieurs par utilisateur
PaymentMethod	Carte tokenisée	Stockage last4 + token Stripe uniquement (<i>PCI-DSS</i>)
Category	Catégorie de produits	Slug SEO, image, ordre d'affichage, statut
Product	Produit du catalogue	Slug, prix HT, TVA, prix TTC, stock, statut, caractéristiques
ProductImage	Images d'un produit	URL Cloudinary, ordre, image principale
Cart	Panier (session ou utilisateur)	CartId UUID pour invités
CartItem	Ligne de panier	Référence produit + quantité
Order	Commande validée	Statut, historique des changements
OrderItem	Ligne de commande	Snapshot prix/nom au moment de la commande
Invoice	Facture associée	N° auto-généré, statut, PDF
CreditNote	Avoir	Référence facture, motif
ContactMessage	Message contact	Email, sujet, message, statut traité
ChatMessage	Message chatbot	Référence conversation, rôle (user/bot)

7.2 — Points de conception critiques

- **Snapshots OrderItem** — Le nom et le prix du produit sont copiés à la création de la commande pour préserver l'historique en cas de modification ultérieure du produit.
- **Stockage carte PCI-compliant** — Seuls last4 et le token Stripe sont conservés. Jamais le PAN complet.
- **Indexation** — sur email (User), slug (Product, Category), createdAt (Order), price/stock (Product), recherche *full-text* sur name/description (Product).

7.3 — Diagramme simplifié (ERD)



∞ Le modèle privilégie l'**intégrité historique** : commandes, factures et avoirs sont immuables après création ; les évolutions passent par création d'avoir plutôt que par mutation.

VIII. Contrats API

L'API REST est documentée via **Swagger UI** en environnement de développement à `/swagger`. Les endpoints sont regroupés par domaine fonctionnel.

8.1 — Authentification

MÉTHODE	ROUTE	AUTH	DESCRIPTION
POST	/api/auth/register	Public	Inscription + envoi e-mail de confirmation
POST	/api/auth/login	Public	Connexion → JWT + refresh token
POST	/api/auth/refresh	Public	Renouvellement du JWT
POST	/api/auth/confirm-email	Public	Validation du compte via token e-mail
POST	/api/auth/forgot-password	Public	Envoi lien reset mot de passe
POST	/api/auth/reset-password	Public	Réinitialisation avec token
POST	/api/auth/logout	Auth	Révocation du refresh token

8.2 — Catalogue

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/api/products	Public	Liste paginée avec filtres/tri
GET	/api/products/{id}	Public	Détail produit
GET	/api/products/slug/{slug}	Public	Détail par slug SEO
GET	/api/products/top/{limit}	Public	Top produits mis en avant
GET	/api/categories	Public	Liste des catégories actives
GET	/api/categories/{slug}	Public	Catégorie + produits associés

8.3 — Panier & commande

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/api/cart/{cartId}	Public	Contenu du panier
POST	/api/cart/{cartId}/items	Public	Ajout d'un article
PATCH	/api/cart/items/{itemId}	Public	Modification quantité
DEL	/api/cart/items/{itemId}	Public	Suppression d'un article
DEL	/api/cart/{cartId}	Public	Vidage du panier
POST	/api/cart/checkout	Auth	Finalisation de la commande
GET	/api/orders	Auth	Historique des commandes
GET	/api/orders/{id}	Auth	Détail d'une commande
POST	/api/orders/{id}/cancel	Auth	Annulation d'une commande
GET	/api/orders/{id}/export	Auth	Export facture PDF

8.4 — Compte utilisateur

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/api/users/me	Auth	Profil de l'utilisateur connecté
PUT	/api/users/me	Auth	Mise à jour du profil
GET · POST · PUT · DEL	/api/users/me/addresses	Auth	Gestion des adresses
GET · POST · PATCH · DEL	/api/users/me/payment-methods	Auth	Gestion des moyens de paiement

8.5 — Contact & IA

MÉTHODE	ROUTE	AUTH	DESCRIPTION
POST	/api/contact	Public	Envoi d'un message de contact
POST	/api/contact/chatbot	Public	Message au chatbot
GET	/api/contact/chatbot/history	Public	Historique de la conversation
POST	/api/translate	Public	Traduction d'un texte
POST	/api/translate/batch	Public	Traduction de plusieurs textes

8.6 — Paiement

MÉTHODE	ROUTE	AUTH	DESCRIPTION
POST	/api/payments/intent	Auth	Création d'un PaymentIntent Stripe
POST	/api/payments/setup-intent	Auth	Enregistrement d'une carte (SetupIntent)

8.7 — Administration

MÉTHODE	ROUTE	AUTH	DESCRIPTION
GET	/api/admin/dashboard	Admin	KPI et indicateurs
GET · POST · PUT · DEL	/api/admin/products	Admin	CRUD produits
POST	/api/admin/products/{id}/images	Admin	Upload images (Cloudinary)
GET	/api/admin/products/export/csv	Admin	Export CSV produits
GET · POST · PUT · DEL	/api/admin/categories	Admin	CRUD catégories
GET · PATCH	/api/admin/orders	Admin	Liste et mise à jour statut commandes
GET · PATCH · DEL	/api/admin/users	Admin	Gestion utilisateurs
GET · PATCH	/api/admin/messages	Admin	Messages et conversations support
GET · PUT	/api/admin/homepage	Admin	Configuration de la page d'accueil
GET · PUT · DEL	/api/admin/invoices	Admin	Gestion factures et avoirs

→ Tous les endpoints /api/admin/* sont protégés par [Authorize(Roles = "admin")]. La documentation interactive complète est exposée à /swagger en développement.

IX. Sécurité technique

9.1 — Authentification & autorisation

- **JWT Bearer** avec expiration courte + refresh token à rotation.
- **Rôles** : user (front office authentifié) et admin (backoffice).
- Endpoints admin protégés par `[Authorize(Roles = "admin")]`.
- Extraction des claims `userId` et `role` côté controllers — sans appel BDD supplémentaire.

9.2 — Validation & robustesse

- Validation des DTO via **FluentValidation** (serveur) + validation réactive Angular (client).
- Mapping des exceptions métier → codes HTTP standardisés via le middleware `ErrorHandling`.
- Logs structurés pour la traçabilité des erreurs.

9.3 — Paiement & données sensibles

- Stripe gère l'intégralité du traitement carte : **aucun PAN complet ne transite par nos serveurs**.
- Seuls `last4` et le token Stripe sont stockés en base.
- Conformité **PCI-DSS** assurée par la délégation à Stripe.

9.4 — Menaces et contre-mesures

MENACE	CONTRE-MESURE
Injection SQL	Requêtes EF Core paramétrées, pas de SQL dynamique.
XSS	Sanitization Angular, validation des entrées côté serveur.
CSRF	JWT Bearer (pas de cookie) — protection native.
Transport	HTTPS obligatoire en production (redirection HTTP→HTTPS).
Secrets	Variables d'environnement — jamais de clés dans le repo Git.
Force brute auth	Limitation des tentatives (<i>à implémenter en v2</i>).
RGPD	Suppression de compte avec avertissement, minimisation des données.

X. Qualité logicielle

10.1 — Stratégie de tests

NIVEAU	OUTILS	PÉRIMÈTRE
Tests unitaires backend	xUnit + Moq + FluentAssertions	Services, validators, entités
Tests unitaires frontend	Vitest + Angular Testing	Services, composants critiques
Tests intégration API	Postman / Newman	Flux auth, panier, commande, admin
Tests e2e	(à définir)	Parcours utilisateur complet
Tests de performance	k6 / JMeter	Endpoints catalogue, recherche, checkout
Tests de sécurité	OWASP ZAP	Scan vulnérabilités
Tests d'accessibilité	Lighthouse + axe DevTools	Pages critiques WCAG 2.1

10.2 — Normes de code

- Architecture **modulaire** et séparation DTO / entités.
- Conventions de nommage : **PascalCase** (C#) / **camelCase** (TypeScript).
- Principe de **moins privilège** pour tous les endpoints.
- Pas de **secrets réels** dans les fichiers versionnés.
- Revue de code **obligatoire via Pull Request** avant merge.

✓ La revue de code n'est pas une formalité : elle constitue la **première barrière qualité**, devant les tests automatisés. Toute PR sans relecture est refusée.

XI. Déploiement & exploitation

11.1 — Environnements

ENVIRONNEMENT	CONFIGURATION	USAGE
Local développement	Docker Compose + hot reload Angular	Développement quotidien
Préprod / Recette	Conteneurs, variables d'environnement	Tests de recette, démonstration
Production	Conteneurs + reverse proxy + SSL	Livraison finale

11.2 — Prérequis

```
Docker Desktop ≥ 24.x
Docker Compose ≥ 2.x
.NET SDK 8.x
```

11.3 — Commandes essentielles

```
# Lancement complet (Docker Compose)
docker compose up -d

# Backend seul
dotnet build
dotnet run --project src/Web/Web.csproj
dotnet test

# Frontend seul
```

11.4 — Supervision recommandée

- Centraliser les **logs applicatifs** (format structuré JSON).
- Monitorer les **erreurs 5xx** et les temps de réponse des endpoints critiques.
- Alertes automatiques : stock bas, échec paiement, messages non traités.

XII. Évolutivité & roadmap

La feuille de route s'organise sur trois horizons. Les évolutions **court terme** sécurisent la livraison MVP ; le **moyen terme** outille la croissance ; le **long terme** ouvre de nouveaux marchés.

12.1 — Court terme — Évolutions prioritaires

- **Durcissement sécurité** : 2FA administrateur, rotation automatique des secrets.
- Compléter le **workflow factures/avoirs** côté backoffice (formulaire modification, historique).
- **Optimisation de la recherche** : full-text indexé + cache résultats fréquents.
- Audit accessibilité **WCAG 2.1** formel avec rapport.

12.2 — Moyen terme

- **API versioning** (/api/v2/...) pour évolution non-destructive.
- **Event-driven** : traitement asynchrone des e-mails, notifications, reporting.
- **Observabilité avancée** : traces distribuées (OpenTelemetry), métriques Prometheus.
- **Performance** : mise en cache Redis pour les données catalogue.

12.3 — Long terme

- Moteur de **recommandations personnalisées** (ML).
- Place de marché **multi-vendeur**.
- **Application mobile native** (React Native / Flutter).



Court terme

Sécuriser le périmètre actuel et lever les écarts identifiés en maintenance.

Moyen terme

Outiller la montée en charge et la fiabilité opérationnelle.

Long terme

Étendre la valeur produit (recommandations, marketplace, mobile).

XIII. Décisions techniques notables

Les choix d'architecture suivants sont structurants pour la solution. Chacun a été pris pour répondre à un compromis explicite entre *maintenabilité*, *conformité* et *maîtrise des coûts*.

DÉCISION	POURQUOI
Clean Architecture	Facilite la maintenance, les tests unitaires et la séparation des responsabilités.
API REST centralisée	Sert le front office et le backoffice depuis la même surface, simplifie les contrats.
Intégrations externes découplées	Via interfaces de service — remplacement facilité (ex. : changer de prestataire paiement).
Conteneurisation Docker	Environnements identiques dev / préprod / prod, onboarding accéléré.
Stripe pour le paiement	Externalisation de la conformité PCI-DSS, fiabilité éprouvée.
Cloudinary pour les images	CDN mondial, transformations à la volée, pas d'infrastructure fichiers à maintenir.
Ollama + Mistral en local	Maîtrise des données, pas de coût API, réponses adaptées au contexte médical.

★ Chaque décision est **réversible** grâce au découplage par interfaces : un changement de prestataire (Stripe → autre, Cloudinary → S3) reste une opération localisée à la couche d'infrastructure.

XIV. Limites & écarts à traiter

Les écarts identifiés ci-dessous sont assumés pour la livraison MVP. Ils sont accompagnés d'un **plan d'action** formel à présenter au jury.

ÉCART	IMPACT	PLAN D'ACTION
Certaines exigences CDC partiellement implémentées	MOYEN	Cf. matrice de conformité 06_matrice_conformite_cdc.md
Performance recherche < 100 ms non prouvée formellement	MOYEN	Campagne k6 sur dataset de 500 produits avant soutenance
WCAG 2.1 non audité formellement	MOYEN	Audit Lighthouse + axe en sprint S8
Checkout invité non implémenté (actuellement auth requise)	ÉLEVÉ	Décision à formaliser et argumenter devant le jury
2FA admin absent	FAIBLE (MVP)	Prévu en évolution prioritaire post-soutenance
Secrets à durcir .env dans repo	ÉLEVÉ	Migration vers variables d'environnement CI/CD avant livraison finale

∴

« La transparence sur les écarts MVP est une posture d'ingénierie : nommer pour traiter, plutôt que masquer pour livrer. »

— Fin du document —

ALTHEA SYSTEMS · DCT V1.0 · AVRIL 2026